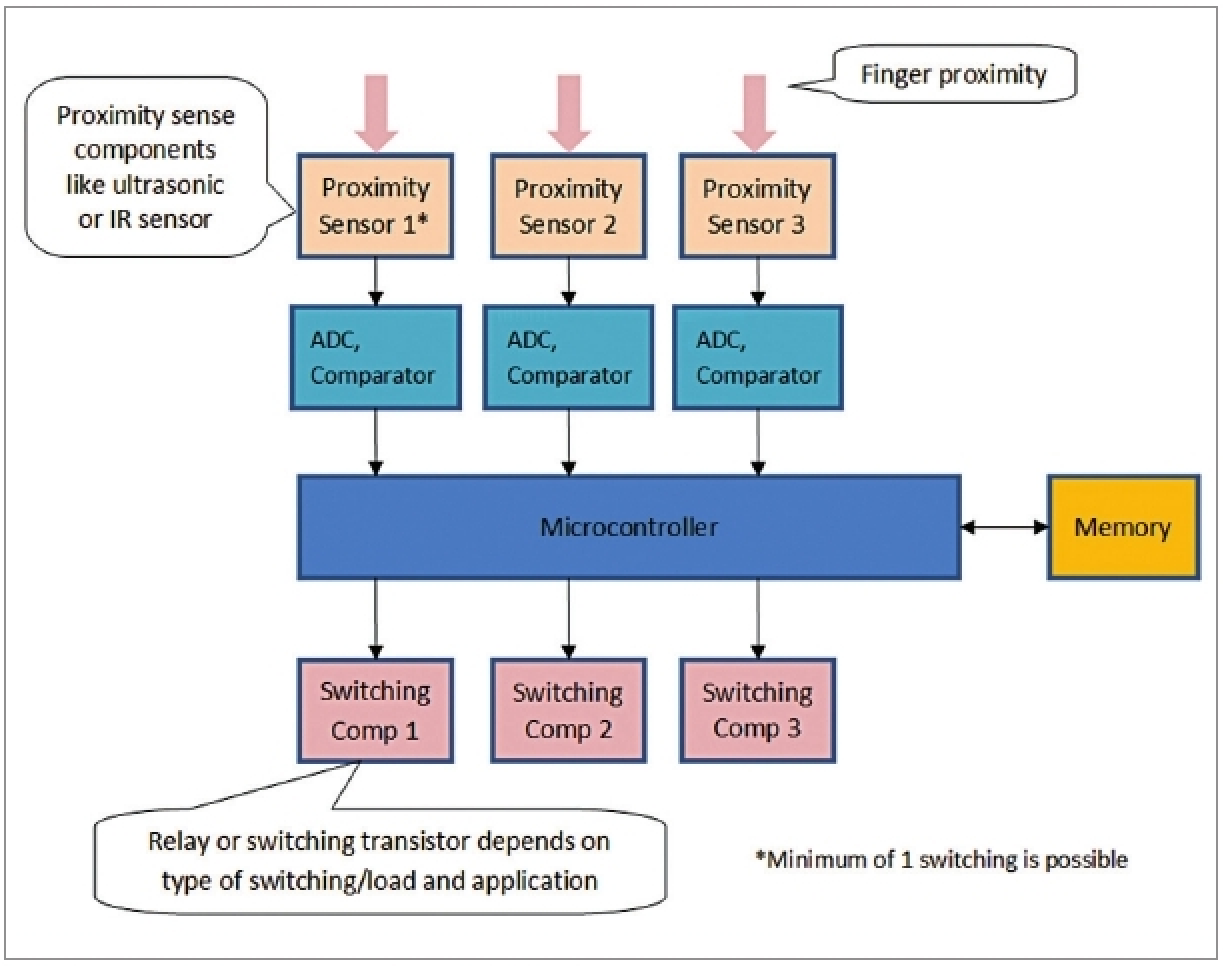

Fig. 1: Proposed scalable hardware architecture

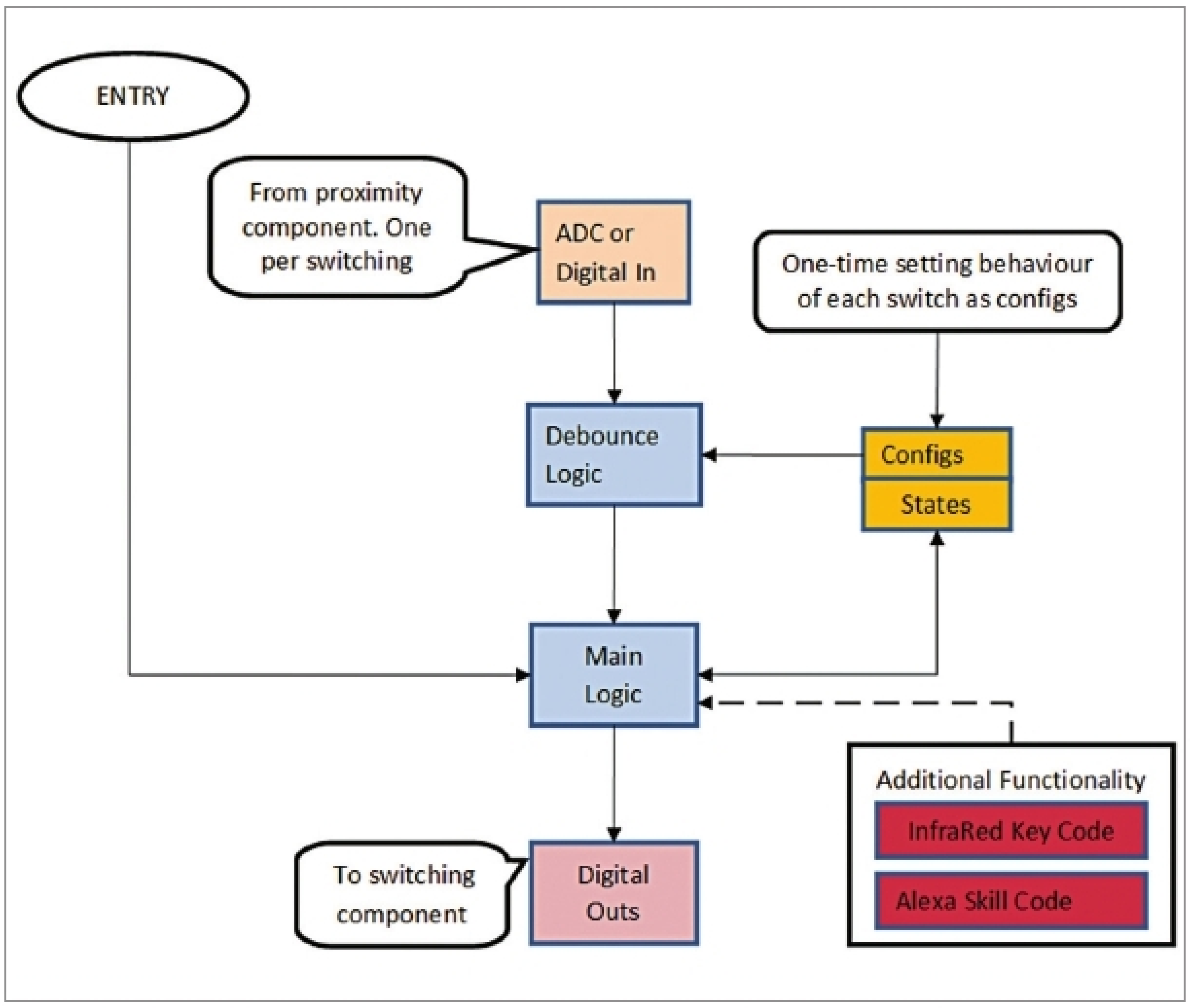

Fig. 2: Proposed event-driven software architecture of touchless sense switching

mode of switching should be possible without changing any component in the system.

- Scope to add or integrate with another technology (Alexa, Wi-Fi, or remote control).

Hardware requirements. New hardware gets added to the existing or new system that facilitates reading a human input without touching the equipment in any form. The hardware setup contains the following:

Main logic. Microcontroller is required.

Input component. Proximity sensors as many as the number of switches are required. These sensors must have good sensitivity and should react only if an object is placed within 1cm proximity.

Switching component. Relay or a high-voltage electronic switching transistor to close high-voltage circuits is required. A low-voltage transistor is good enough for closing low-voltage circuits.

Memory to store configurations and states of each switching is required.

Sensors for sensing the raw event

The following technologies can be applied to realise the solution of touchless sense panel:

1. Ultrasonic proximity sensors. Each button on panels can be replaced by one sense module/area. Each sense module/area acts as an ultrasonic transmitter and receiver.

2. Infrared proximity sensors. Each button on the panels can be replaced by one sense module/area. Each sense module/area has one IR transmitter and one IR receiver.

The functional logic of these proximity sensors is like the proximity sensors that are embedded on mobile phones that switch off the display during a call when the mobile phone is placed next to the ear. When the mobile phone is moved away from the ear/object, the display gets switched on. Similar logic has been used here with better sensitivity and tuning so that only very close proximity (~1cm) objects get sensed.

Proposed scalable hardware architecture is shown in Fig. 1.

Software requirements. The software running in a microcontroller should do the following:

1. Read the raw outputs from proximity sensors and process the raw signals (analogue or digital) when a finger is placed near proximity sensors.
2. Apply debouncing logic.
3. Set output based on the results of debounce logic.
4. In case the switch type is on-off, save the state of this switch in memory so that it can be restored after power resumption in case there is a power failure.